

Warszawa, data 28.05.2026

STUDIA PODYPLOMOWE – TESTER AUTOMATYZUJĄCY W SELENIUM



**AKADEMIA
LEONA KOŹMIŃSKIEGO**

**PROJEKT: AUTOMATYZACJA TESTÓW APLIKACJI WEBOWEJ W
JAVA Z WYKORZYSTANIEM BIBLIOTEKI WEBDRIVER**

Adres strony internetowej testowanego projektu: pepco.pl

[] Zastosowanie Data Driven Testing

[] Zastosowanie wzorca Page Object

Autor:

Anna Janaszek

Ocena:

1 Opis projektu

Celem projektu było przygotowanie automatycznych testów funkcjonalnych strony internetowej Pepco z wykorzystaniem Selenium WebDriver w języku Java.

Wybór strony Pepco nie był przypadkowy. Na co dzień zawodowo pracuję z klientem Pepco w obszarze data management platform (DMP/CDP) oraz aktywacji danych w platformach reklamowych takich jak DV360, Google Ads i Meta. Dzięki temu projekt miał również wymiar praktyczny związany z jakością działania ścieżek użytkownika oraz poprawnością implementacji elementów odpowiedzialnych za odkładanie danych marketingowych i analitycznych.

Strona Pepco nie posiada klasycznego procesu e-commerce z finalizacją zakupu. Z tego powodu projekt został skoncentrowany na:

TC_01: CollectionTest - Sprawdzenie przejścia z kolekcji produktów na stronę produktu oraz przejścia do zasad zwrotu.

TC_02: DmpEventsTest - Sprawdzenie działania requestów DMP/Mediarithmics po akceptacji cookies.

TC_03: HomeCarouselPinnedProductTest - Sprawdzenie działania karuzeli oraz produktu przypiętego do zdjęcia.

TC_04: NewsletterFakerTest - Sprawdzenie formularza newslettera z użyciem danych generowanych przez DataFaker.

TC_05: PepcoClubTest – Sprawdzenie poprawności działania linków App Store i Google Play.

TC_06: RandomCityFromFakerTest – Sprawdzenie działania lokalizatora sklepów dla losowego miasta.

TC_07: StoreLocatorCsvTest – Sprawdzenie wyszukiwarki sklepów dla danych z pliku CSV.

TC_08: TrendyTest – Sprawdzenie działania strony „Hity tygodnia”.

Zastosowane techniki:

- Selenium WebDriver
- Java
- JUnit 5
- Maven
- Page Object Model
- Data Driven Testing
- DataFaker
- WebDriverWait

2 Przypadki testowe

Kolekcja, produkt, informacje o zwrotach.

ID Testu	TC_01
Opis Testu	Sprawdzenie przejścia z kolekcji produktów na stronę produktu oraz przejścia do zasad zwrotu.
Kroki do wykonania	1. Test pobiera dane z CollectionData#returnPolicyCollections. 2. Otwiera wskazana kolekcje, np. /collections/dom/oswietlenie albo /collections/dziecko/chlopiec-92-134. 3. Akceptuje cookies. 4. Klika trzeci produkt na stronie kolekcji. 5. Sprawdza, czy URL zawiera /products/. 6. Na stronie produktu klika informacje "30 dni na zwrot". 7. Otwiera regulamin lub strone zasad zwrotu. 8. Sprawdza, czy adres URL zawiera oczekiwany fragment. 9. Czeka az strona sie ustabilizuje. 10. Wykonuje screenshot dla danego wariantu testowego.
Oczekiwany rezultat	- czy kolekcja zawiera co najmniej trzy produkty, - czy kliknięcie produktu otwiera stronę produktu, - czy z produktu da się przejść do informacji o zwrotach, - czy otwierana jest poprawna strona/regulamin zwrotow.

Diagnostyka requestow Mediarithmics

ID Testu	TC_02
Opis Testu	Sprawdzenie działania requestów DMP/Mediarithmics po akceptacji cookies.
Kroki do wykonania	1.Otwórz stronę Pepco. 2. Zaakceptuj cookies. 3. Odśwież stronę. 4. Pobierz logi performance Chrome. 5. Zweryfikuj requesty Mediarithmics.
Oczekiwany rezultat	Requesty DMP zawierają site_token=pepco-24-pl.

Karuzela, CTA, produkt z pinu

ID Testu	TC_03
Opis Testu	Sprawdzenie działania karuzeli oraz produktu przypiętego do zdjęcia.
Kroki do wykonania	1. Otwiera strone glowna pepco.pl.

	2. Czeki na ładowanie strony. 3. Akceptuje cookies. 4. Przechodzi po karuzeli na stronie głównej. 5. Klika CTA z sekcji na stronie głównej. 6. Klika pin produktu widoczny na zdjęciu. 7. Klika produkt pokazany w panelu pinu. 8. Czeki na przejście do strony produktu. 9. Sprawdza, czy URL zawiera /products/. 10. Wykonanie screenshot home_carousel_pinned_product.
Oczekiwany rezultat	- czy karuzela jest interaktywna, - czy CTA na stronie głównej działa, - czy pin produktu na zdjęciu otwiera panel produktu, - czy kliknięcie produktu z pinu prowadzi do strony PDP.

Wpisanie adresu e-mail do formularza newslettera

ID Testu	TC_04
Opis Testu	Sprawdzenie formularza newslettera z użyciem danych generowanych przez DataFaker.
Kroki do wykonania	1. Otwiera stronę główną pepco.pl. 2. Czeki na ładowanie strony. 3. Akceptuje cookies. 4. Generuje adres e-mail przez DataFaker. 5. Wpisuje e-mail do formularza newslettera. 6. Wykonuje screenshot
Oczekiwany rezultat	Losowy mail z DataFaker w formularzu newslettera.

Linki do App Store i Google Play

ID Testu	TC_05
Opis Testu	Sprawdzenie linków do aplikacji Pepco w App Store i Google Play.
Kroki do wykonania	1. Test pobiera dane z PepcoClubData#appStoreLinks. 2. Otwiera stronę /pepco-club. 3. Akceptuje cookies. 4. Sprawdza, czy URL zawiera pepco-club. 5. Sprawdza, czy tytuł strony Pepco Club jest widoczny. 6. Klika link App Store albo Google Play. 7. Przelicza się na najnowsza kartę przeglądarki. 8. Czeki na ładowanie strony sklepu. 9. Sprawdza, czy URL zawiera oczekiwany adres: apps.apple.com, play.google.com. 10. Wykonuje screenshot strony sklepu.

Oczekiwany rezultat	Co jest sprawdzane: - czy strona Pepco Club sie otwiera, - czy linki do sklepow aplikacji sa klikalne, - czy prowadzi do poprawnych zewnetrznych stron, - czy dziala obsluga nowej karty.
---------------------	---

Losowe miasto z DataFaker w lokalizatorze

ID Testu	TC_06
Opis Testu	Sprawdzenie wyszukiwarki sklepow z losowym miastem wybranym przez DataFaker.
Kroki do wykonania	1. Otwiera strone /store-locator. 2. Akceptuje cookies. 3. Losuje miasto z listy: Warszawa, Krakow, Gdansk, Wroclaw, Poznan, Lodz. 4. Wypisuje testowane miasto w konsoli. 5. Wpisuje miasto w wyszukiwarke sklepow. 6. Sprawdza, czy pole wyszukiwania nie jest puste. 7. Sprawdza, czy pole zawiera szukane miasto. 8. Sprawdza, czy pojawil sie wynik wyszukiwania dla miasta. 9. Otwiera pierwszy sklep z wynikow dla tego miasta. 10. Ustawia sklep jako "Twoj sklep". 11. Sprawdza, czy URL po kliknieciu nie jest pusty. 12. Wykonuje screenshot <code>random_city_favorite_store</code>.
Oczekiwany rezultat	- dzialanie wyszukiwarki sklepow, - obsluga danych generowanych przez Faker, - otwieranie szczegolow sklepu, - ustawienie sklepu jako ulubionego.

Miasta z pliku CSV w lokalizatorze sklepow

ID Testu	TC_07
Opis Testu	Sprawdzenie formularza newslettera z uzyciem danych generowanych przez DataFaker.
Kroki do wykonania	1. Test pobiera dane z pliku <code>src/test/resources/test_data/store_locator_data.csv</code>. 2. Dla kazdego miasta otwiera strone /store-locator. 3. Akceptuje cookies. 4. Wpisuje miasto w wyszukiwarke sklepow.

	5. Sprawdza, czy pole wyszukiwania nie jest puste. 6. Sprawdza, czy pole zawiera oczekiwany tekst miasta. 7. Sprawdza, czy pojawiły się wyniki wyszukiwania dla miasta. 8. Otwiera pierwszy sklep z wyników dla danego miasta. 9. Ustawia sklep jako "Twój sklep". 10. Wykonuje screenshot <code>store_locator_csv_favorite_nazwa</code> . 11. Wypisuje testowane miasto w konsoli.
Oczekiwany rezultat	- data driven testing z plikiem CSV, - wyszukiwanie sklepów dla wielu danych, - otwieranie szczegółów sklepu, - ustawienie sklepu jako ulubionego.

Strona oferty Hity Tygodnia/Trendy i produkty

ID Testu	TC_08
Opis Testu	Sprawdzenie strony "Oferta gazetkowa" i produktów widocznych w tej sekcji.
Kroki do wykonania	1. Otwiera <code>/trendy/oferta-gazetkowa</code> . 2. Akceptuje cookies. 3. Sprawdza, czy URL zawiera ofertę gazetkową. 4. Wykonuje screenshot <code>trendy_page</code> .
Oczekiwany rezultat	- wejście na stronę oferty gazetkowej, - widoczność produktów, - przejście z listy produktów na stronę produktu.

3 Kod w Java

https://github.com/annajanaszek86-waw/TAS5_zal_AJanaszek

4 Wnioski

Projekt pozwolił na praktyczne wykorzystanie Selenium WebDriver, Java oraz JUnit 5 do automatyzacji testów rzeczywistej strony internetowej działającej produkcyjnie.

Największym wyzwaniem podczas realizacji projektu okazała się stabilność testów uruchamianych na dynamicznej stronie komercyjnej. W trakcie prac konieczne było ograniczenie zależności od kruchych selektorów, dynamicznych elementów SVG oraz elementów zależnych od zewnętrznych skryptów marketingowych.

Projekt pokazał również, że podczas automatyzacji testów dla rzeczywistych aplikacji webowych bardzo ważne jest:

- ograniczanie zależności od dynamicznego HTML,
- tworzenie bardziej odpornych asercji,
- uwzględnianie niestabilności systemów zewnętrznych, np. tagów marketingowych lub mechanizmów antyspamowych.

Dzięki zastosowaniu Page Object Model kod projektu został uporządkowany i łatwiejszy do utrzymania. Wykorzystanie Data Driven Testing umożliwiło testowanie wielu scenariuszy przy użyciu różnych zestawów danych bez powielania kodu.

Dodatkową wartością projektu było wykorzystanie wiedzy praktycznej związanej z obszarem digital marketingu oraz Data Management Platform. Testy diagnostyczne DMP pozwoliły sprawdzić poprawność działania requestów marketingowych i trackingowych, co jest istotnym elementem jakości wdrożeń analitycznych i reklamowych.